

MoDELib2 Guide

Matthew Maron, Liam Myhill, Hyunsoo Lee

March 2024

Contents

1	Installing MoDELib2 Dependencies	2
1.1	MacOS	2
1.1.1	Installing Dependencies	2
1.2	Linux	3
1.2.1	Installing Dependencies	3
1.3	Windows	5
1.3.1	Installing Dependencies	5
2	Installing Visualization Tool Dependencies	6
2.1	Compiling QT with VTK via CMake GUI	7
2.1.1	Resolving Deprecated AGL Framework Errors in VTK on macOS	9
3	Compiling MoDELib2	11
3.1	Accessing the Repository	11
3.2	Compiling MoDELib2 Tools	11
3.2.1	SDK version for Mac	12
3.2.2	Python and pybind11 mismatch	12
4	Running MoDELib2	13
4.1	Input Files	13
4.2	MicrostructureGenerator	13
4.3	DDomp	14
4.4	DDqt	14
5	Simulation Tutorials	15
5.1	Periodic Simulations	15
5.1.1	Dislocation Dipoles	15
6	Additional Features	18
6.1	Mesh Generation	18
7	Understanding the Code	19
A	Text Editors	20
B	Web Addresses for Software	21

Chapter 1

Installing MoDELib2 Dependencies

In order to efficiently use MoDELib2, a text editor is necessary. One can use XCode, Visual Studios, TextEdit, Vim, etc. as detailed in Appendix A. Details on the versions of needed software are shown in Appendix B.

1.1 MacOS

For MacOS, it is first necessary to download XCode from the Mac App Store. Once this has been installed, the command lines tools will be necessary. A terminal can be opened by simply typing "Terminal" into the spotlight search. Once XCode has been downloaded, to install the command line tools, simply type the following lines into the terminal

```
$ xcode-select --install
```

A prompt will then appear to accept the download of the command line tools. Now, to manage the installation of other MoDELib2 dependencies, it is recommended to install MacPorts as a package manager over other package managers like homebrew. Install MacPorts from the website: <https://www.macports.org/install.php>. Under the "Installing MacPorts" tab, please install MacPorts for your version of the Mac operating system. Now other packages can be installed using MacPorts with the following line in the command shell:

```
$ sudo port install $package name$
```

For example, MoDELib2 features a modern CMake-based build system, as opposed to the Makefile-based build system of MoDELib1. To install cmake via MacPorts, simply type the following into the command shell,

```
$ sudo port install cmake
```

Now all dependencies to use cmake for creating make files for MoDELib2 have been installed in the /opt/local directory. The /opt/local directory is the default location for installations with MacPorts, unless otherwise specified during the installation of MacPorts.

1.1.1 Installing Dependencies

Installing MacPort Dependencies

To compile dislocation dynamics programs without any issues, you need to have the necessary library installed on your system. This includes eigen, fftw, pybind11, boost, VTK as well as QT. With MacPorts eigen, fftw, and pybind can be easily installed as follows. Open a terminal window and to install eigen type in the terminal,

```
$ sudo port install eigen3
```

for fftw type,

```
$ sudo port install fftw-3
```

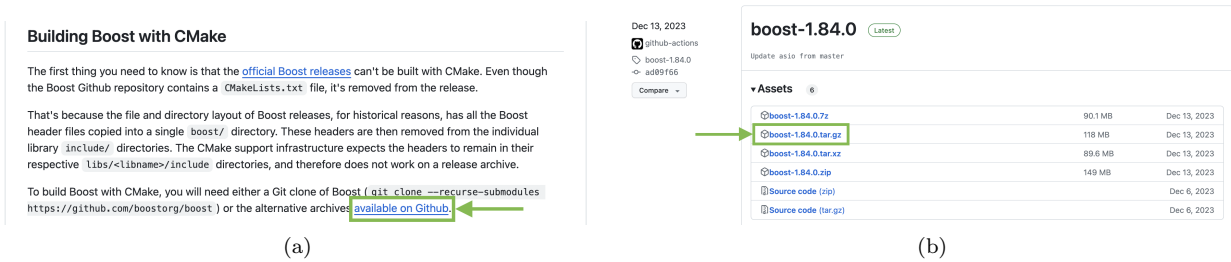


Figure 1.1: (a) Boost GitHub Page, (b) boost installation

for OpenMP type

```
$ sudo port install libomp
```

finally for pybind type,

```
$ sudo port install pybind11
```

which should also install the necessary python dependencies. All dependencies will now be installed in the /opt/local/ folder of your MacOS system.

Boost

First visit the boost GitHub page: <https://github.com/boostorg/cmake?tab=readme-ov-file>. Scroll down and click the hyper-link shown in Fig. 1.1a. Next download the zipped file from Fig. 1.1b and open the zipped file from the downloads folder. Once downloaded open a terminal inside the boost directory and perform the following terminal commands,

```
$ mkdir build
$ cd build
$ cmake ..
$ sudo make install
```

Once the binaries have been installed, the boost directory is no longer needed.

SuiteSparse

SuiteSparse is a suite of sparse matrix algorithms authored or co-authored by Tim Davis, Texas A&M University. It accelerate the solution of sparse linear system of equations compared to built-in Eigen solvers.

Download from <https://github.com/DrTimothyAldenDavis/SuiteSparse>. To install isse the following terminal commands:

```
$ mkdir -p build && cd build
$ cmake ..
$ cmake --build .
$ sudo cmake --install .
```

1.2 Linux

1.2.1 Installing Dependencies

This section details the installation of MoDELib2 using a Linux OS. It should be noted that the author is running Ubuntu 20.04.6 LTS, although other distros of the Linux OS can be utilized. It is assumed that the package manager utilized throughout the process is apt; however, any package manager such as dnf that offers the necessary dependencies should suffice. In the event that the above packages are not available on the utilized

Linux distribution, it may be necessary to find which versions of specific packages are available to the user. To find available packages related to a *keyword*, the following line will list all available packages:

```
$ apt search keyword
```

To download the necessary dependencies type in the terminal, for example:

```
$ sudo apt install cmake
```

will install the cmake compiling package in the default directory as specified by your OS configuration. For standard Ubuntu Linux, the default directory is `/usr/local`. The same procedure can be used to download all of the dependencies of MoDELib. It is worth noting that the text editor Vim is also available on Linux OS, and the instructions provided in Sec. A are applicable to Linux users. For convenience, the following commands have been provided to install the MoDELib2 dependent libraries on Linux OS.

```
$ sudo apt install eigen3
```

```
$ sudo apt install fftw-3
```

```
$ sudo apt install libomp
```

```
$ sudo apt install python3
```

```
$ sudo apt install pybind11
```

OpenGL is required for many visualization-based applications, and can be installed by passing the following arguments into the command line.

```
$ sudo apt install libglew-dev libgl sdl
```

Boost

First visit the boost GitHub page: <https://github.com/boostorg/cmake?tab=readme-ov-file>. Scroll down and click the hyper-link shown in Fig. 1.1a. Next download the zipped file from Fig. 1.1b and open the zipped file from the downloads folder. Once downloaded open a terminal inside the boost directory and perform the following terminal commands,

```
$ mkdir build
$ cd build
$ cmake ..
$ sudo make install
```

Once the binaries have been installed, the boost directory is no longer needed.

SuiteSparse

SuiteSparse is a suite of sparse matrix algorithms authored or co-authored by Tim Davis, Texas A&M University. It accelerate the solution of sparse linear system of equations compared to built-in Eigen solvers.

Download from <https://github.com/DrTimothyAldenDavis/SuiteSparse>. To install isse the following terminal commands:

```
$ mkdir -p build && cd build
$ cmake ..
$ cmake --build .
$ sudo cmake --install .
```

1.3 Windows

1.3.1 Installing Dependencies

This section targets the users that need to use windows to run DDqt. In order to do that, the first step is to download and install visual studio 2022, cmake, vtk and qt for windows.

- vs community 2022 (<https://visualstudio.microsoft.com/downloads/>)
- cmake-3.29.2-windows-x86_64.msi (<https://cmake.org/download/>)
- vtk-9.3.tar.gz (<https://vtk.org/download/>)
- qt-online-installer-windows-x64-4.8.0.exe (<https://www.qt.io/download-qt-installer>)

Building vtk

first step is to extract vtk files from VTK-9.3.0.tar using a powershell.

- open a windows PowerShell as administrator
- go to downloads. cd . Downloads
- tar - xvf vtk-9.3.tar.gz

open cmake programm and browse to the vtk in downloads C:/User/xxx/Downloads/vtk-9.3.0. Then, assign C:/User/xxx/Downloads/vtk-9.3.0/build to browse build box and click the configure button. A dialog box will open asking to specify the generator for the project. In the present case specify visual studio 17 2022 and click finish.

Use of MoDELib2 with Windows currently limited.

Chapter 2

Installing Visualization Tool Dependencies

For visualization with MoDELib2, QT must be linked with VTK using the CMake GUI, which will be shown in the next section. This section is focused on installing the proper and necessary software to use MoDELib2 visualization. Fig. 2.1 highlights the steps of downloading DDqt, and will be discussed here step by step.

First visit: <https://www.qt.io/download>. Then, following the green arrows, first select "Download Open Source" as shown in Fig. 2.1a. Next, scroll down to the screen shown in Fig. 2.1b, and select "Download the Qt Online Installer". Once the online installer has been downloaded, open the QT maintenance for installing the correct packages and components with the Qt download. After logging in make sure to select a "Custom Installation" as shown in Fig. 2.1c. Once inside the custom installation follow the selections from Fig. 2.1d. First ensure to select the correct version of Qt, in this case 6.6.2, is selected.

Now that the proper Qt version is to be installed we can begin to remove unnecessary components to reduce download size. This is done with reference to Fig. 2.1d. Open up the drop down menu for Qt 6.6.2, and remove all of the components for Android, iOS, etc. in this menu; only leaving the macOS box checked off. Now the "Additional Libraries" drop down can be selected, and only Qt Charts is needed, all other components can be un-selected. Finally, select the drop down menu for "Developer and Designer Tools" and un-select CMake, as well as Ninja, as these tools are not needed. This will greatly reduce the amount of space Qt will take up on your hardware, and the entire package should be less than 10 Gb after downloading.

Once Qt has been properly installed, VTK should be downloaded. Please install the files directly from the vtk website, and avoid using MacPorts or any other package managers for this installation. Download VTK source code from <https://vtk.org/download/> and find the zipped ".tar.gz" file of the version desired. In this example, VTK-9.3.0 is being downloaded. Once downloaded unpack the archive file by double clicking and a folder labelled VTK-9.3.0 should now appear in the downloads folder. Leave this here, as once the binaries have been created, this can be removed. To create the binaries and properly link them to Qt, please see the next section of this report.

In order to link VTK and QT, a CMake GUI is also needed. Please visit, <https://cmake.org/download/>, to find the correct version. Do Not download a "Source distribution", but instead scroll down to find a "Binary distribution" suitable to your macOS version. In this example macOS Sonoma is being used so "cmake-3.29.0-rc3-macos-universal.dmg" for macOS 10.13 or later has been downloaded. Once downloaded open the dmg file and drag the CMake GUI icon to the Applications folder. With all of QT, VTK, and the CMake GUI, we are now ready to link VTK to Qt for visualization purposes in MoDELib2 which will be discussed in the following section.

Note for Linux Users

Note on X11. If you plan to connect remotely to your Linux workstation, you may need to install X11 *before* installing Qt. To install the X11 requirements, follow the instructions on this page: <https://doc.qt.io/qt-6/linux-requirements.html>.

To download QT go to the website <https://www.qt.io/download-dev>. From the download page select the Community User, **Explore QT Community Edition**. Scroll down to the Looking for QT Binaries? section as shown in Fig. 2.1b and select **Download the QT Online Installer**. From here select Linux and a .run file will be downloaded. Then in the downloads folder of the terminal perform the following commands

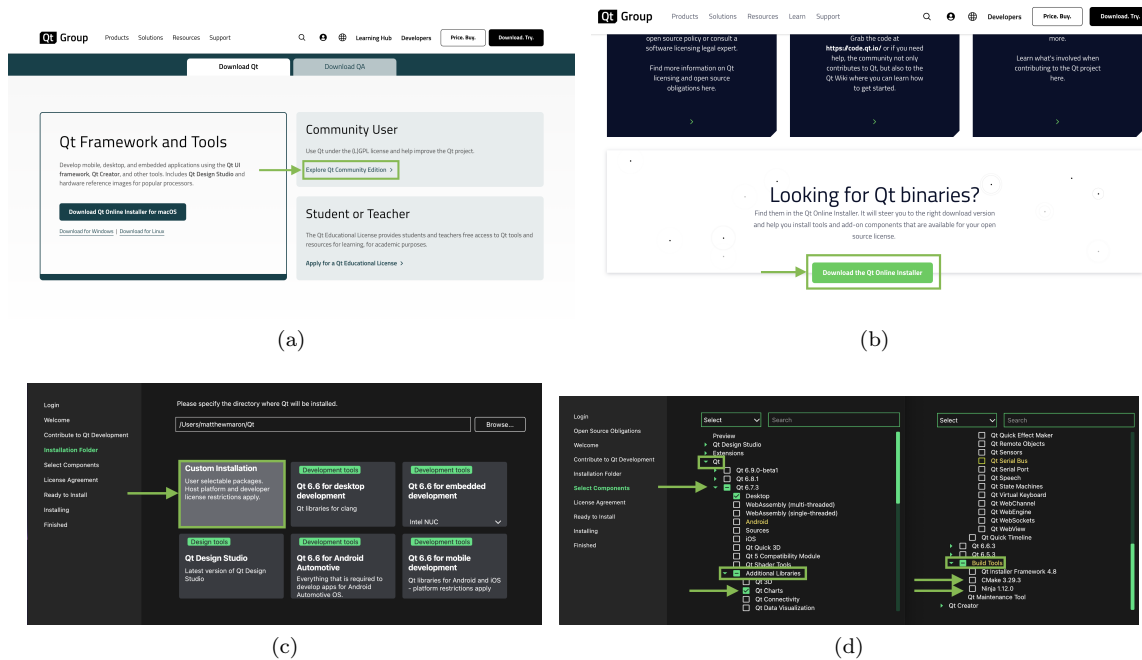


Figure 2.1: (a) Finding the open source version of Qt, (b) Finding the Qt Online Installer, (c) Ensuring installation in the proper location and selecting a Custom Installation, (d) Selecting the desired version of Qt and un-selecting the components not necessary for visualization in MoDELlib2.

```
$ chmod +x $FileName$.run
$ ./FileName$.run
```

The QT installer will be opened, and now the rest of Section 2 can be followed. Be sure not to install the Mac version of QT but the QT necessary for Linux. The previously referenced Section 2 also includes instruction on downloading VTK as well as the CMake GUI. Once this section is followed and completed, then go to section 2.1 to compile QT with VTK in order to use the visualization tool associated with DDQT.

2.1 Compiling QT with VTK via CMake GUI

To link VTK with Qt for visualization, the following steps are provided with reference to Fig. 2.2.

Step 1: Following the download of QT and VTK, first open the CMakeGUI. This should prompt a screen identical to the background window of Fig. 2.2a. Now, create a build directory inside of the VTK folder, this can be done with the terminal commands

```
$ cd ~/VTK-9.3.0/
$ mkdir build
```

With the build folder created in VTK, return to the CMakeGUI, and for the prompt "Where is the source code:" Use the menu in the upper right hand corner of the window labeled "Browse Source." Find the VTK-9.3.0 folder and use this as the source code. Next in the prompt for "Where to build the binaries:" similarly select "Browse Build.." and find the build folder created inside of VTK for this directory. Once the Source and Build directories have been specified select "Configure" in the bottom left hand corner of the window. This will try and build the binaries but errors will occur at this step.

Step 2: Once the CMake GUI has finished attempting to build the binaries, use the search bar to make the first necessary changes to the configuration. Please check the "Grouped" and "Advanced" options to the right of the search bar to make finding and visualizing the changes needed easier, as shown in Fig. 2.2b. Type "Qt" into the search bar and select the drop down menu for VTK. In the drop down menu change the `VTK_GROUP_ENABLE_Qt` to a value of Yes. This will let CMake know you would like to link the VTK library while creating Qt. Once finished, click configure again, and more errors will occur.

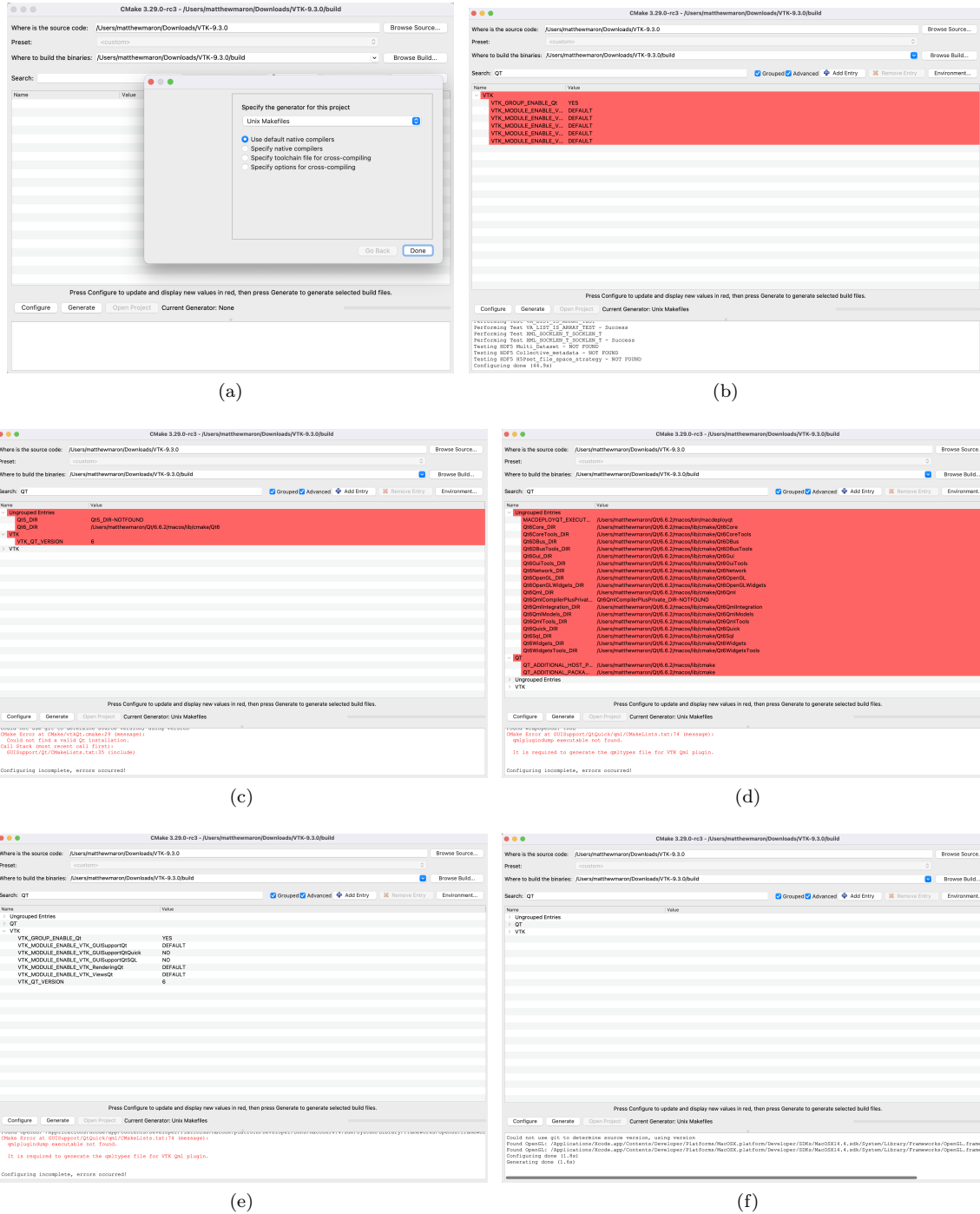


Figure 2.2: Visual aids in linking VTK and Qt with the CMake GUI discussed in Sec. 2.1 (a) Step 1: Opening the CMake GUI and passing the VTK Source and Build directories, (b) Step 2: Enabling the linking between VTK and Qt in the CMake GUI, (c) Step 3: Pointing CMake to the directory of the Qt installation and specifying the version of Qt being used, (d) Step 4: Ensuring correct paths to package directories and providing the location for Host Packages, (e) Step 5: Turning off QtQuick and QtSQL Support not necessary for compiling MoDELib2 visualization software, (f) Step 6: Configuration and Generation complete, ready for installing binaries.

Step 3: Following now Fig. 2.2c, keep "Qt" inside the search bar and the "Grouped" and "Advanced" options checked. First open the "Ungrouped Entries" drop down menu. Under this menu "Qt6_DIR" will allow you to give CMake the correct path to the installed Qt. Please ensure this is to the Qt version inside the lib folder with CMake. For example, `/Qt/6.6.2/macos/lib/cmake/Qt6/` would be a proper path to the Qt directory. Next in the VTK drop down menu, pass the version of Qt you are using in the `VTK_QT_VERSION` section. For this example Qt6 is being used, so a value of 6 was chosen. Once finished, click configure again to be followed by some more errors when trying to build the binaries.

- Step 4: In this step, many entries should appear red as shown in Fig. 2.2d under the "Ungrouped Entries" drop down tab. If any of the directories for tools shown in Fig. 2.2d are missing, this can be resolved by finding the directory inside Qt, and entering it into the value field. For example if the `CoreToolsDir-NOTFOUND` then: `CoreToolsDir → /Qt/6.6.2/macos/lib/cmake/Qt6CoreTools/` specifies where to find local files. Again this may need to repeat for other directories. QML and SQL are not needed which will be discussed in the next section, so their directories are not necessary. After checking the "Ungrouped Entries" drop down menu, then go to the "QT" drop down menu. In the `QT_ADDITIONAL_HOST_PACKAGES` menu please place the location of the cmake dependencies inside the Qt directory. This is the same path as in step 3, without pointing to the Qt directory. For example, `/Qt/6.6.2/macos/lib/cmake/` would be a proper path. For `QT_ADDITIONAL_PACKAGES`, please use the same path. Once finished click configure once more, and errors involving SQL or QML may now appear.
- Step 5: If errors involving SQL or QML appear when configuring in Step 4, please select the drop down menu for VTK as shown in Fig. 2.2e. The "Enable Qt" section should already have a value of Yes, and the "Qt version" should already be specified following from Step 2. Please click on the `ENABLE_VTK_GUI_SupportQtQuick` and `ENABLE_VTK_GUI_SupportQtSQL` and set each of these values to No. Once finished please configure one last time.
- Step 6: After following Steps 1-5, and having downloading the correct dependencies, configuration should be completed as shown in Fig. 2.2f. Once configuration is done, the binaries can now be generated using the "Generate" command in the bottom left of the window. Once finished, all the necessary steps involving the CMake GUI are completed and it may be closed. Now, a terminal can be opened in the build folder in VTK-9.3.0, and the following commands can be performed,

```
$ make -j6 (or however many cores desired)
$ sudo make install
```

Once finished the VTK folder is no longer needed as the binaries have been installed on your hardware.

Resolving Template Compilation Issues in macOS: Switching to Clang++-MP-18

While attempting to compile a C++ project using CMake on macOS—which includes VTK and Qt6 dependencies—a critical compilation issue arose when using the default Apple Clang compiler. Specifically, the build failed during instantiation of a complex C++ template class, due to missing or incomplete support in the system Clang version provided by Xcode. This issue commonly appears when the compiler fails to expand or instantiate a template class correctly, despite valid syntax and logic. To resolve this issue, the compiler was switched from the system-default Clang to the LLVM 18 toolchain provided by MacPorts. The 'clang++-mp-18' compiler offers full C++20/C++23 template support and tracks upstream LLVM behavior more closely, ensuring compatibility with templated constructs used in VTK and custom code. When using a custom Clang compiler with CMake, additional toolchain binaries must also be specified explicitly, or the configuration will fail.

Install the full toolchain via MacPorts:

```
$ sudo port install clang-18
```

Use the following commands to locate the necessary binaries:

```
$ which clang++-mp-18      # => /opt/local/bin/clang++-mp-18
$ which llvm-ar-mp-18      # => /opt/local/bin/llvm-ar-mp-18
$ which llvm-ranlib-mp-18  # => /opt/local/bin/llvm-ranlib-mp-18
$ which clang-scan-deps-mp-18 # => /opt/local/bin/clang-scan-deps-mp-18
```

When using the CMake GUI, manually assign each of these variables. A visual example of where these entries must be corrected is shown in Figure 2.3.

2.1.1 Resolving Deprecated AGL Framework Errors in VTK on macOS

Following updates to XCode or the macOS SDK, VTK may fail to compile with Qt support due to the deprecated Apple OpenGL framework `AGL.framework`. A typical linker error appears as:

Chapter 3

Compiling MoDELib2

3.1 Accessing the Repository

- Step 1: Prior to using MoDELib2, ensure you have installed either a LINUX/UNIX operating system (OS), or have access to one via a virtual machine. Visit Chapt. 1.1 for ensuring to install the proper dependencies if using macOS and Chapt. 1.2 if using a Linux operating system.
- Step 2: Ensure the following software is installed prior to installation of MoDELib2. They will be required to compile the executable functions. Version numbers provided where necessary. Ensure codes are compiled and installed when necessary, and maintain easy access to the directory in which the codes are stored.

Software	Version
CMake	(version 3.23.0 or later)
Eigen	(version 3.1 or later)
Qt6	(versions 5 or 6)
FFTW3	(-)
VTk	(version 9.1 or later)
Python	(version 3.11 or later)

- Step 3: Using a browser of your choice, go to <https://github.com/giacomopo/MoDELib2> to access the link to the git repository to clone the library:

```
$ https://github.com/giacomopo/MoDELib2
```

It should be noted that multiple branches of the MoDELib2 may be available still. This walk-through will assume the user intends to download the currently active branch, MoDELib2.

- Step 4: In a directory of your choice (file-system examples provided in brackets will assume home directory), create a new directory to store MoDELib.

```
$ mkdir MoDELib2
$ cd MoDELib2
$ git clone https://github.com/giacomopo/MoDELib2.git
```

3.2 Compiling MoDELib2 Tools

1. Create microstructureGenerator, DDomp and DDqt applications. To create the tools microstructureGenerator, DDomp and DDqt, the cmake commands should be run within a build folder created in the main MoDELib2 directory. First, go to the CMakeLists.txt in the main directory and specify the paths to the installed Eigen library and the Qt folder. This is done as follows

```
$ set(EIGEN3_INCLUDE_DIRS /opt/local/include/eigen3)
$ list(APPEND CMAKE_PREFIX_PATH "/Users/matthewmaron/Qt/6.6.2/macos/lib/cmake")
```

Next, to create the MoDELib2 tools, create a build folder. The "cmake.." calls installed cmake compiler to reference CMakeLists in the above directory, then writes all make files in build directory. "Make" then reads the makefiles and compiles the executable. Perform the commands

```
$ mkdir build; cd build
$ cmake ..
$ make -j$Number of processors$
```

After compilation, the executables called "DDomp", "microstructureGenerator", and "DDqt" will be created in the MoDELib2/build/tools/ directory in individual folders.

MoDELib2 also now generates a python library along with the main tools. In this way MoDELib2 can be controlled purely from python, without the need to create executables. MoDELib2 also has functionality to embed python code into the main code, for using custom mobility functions or machine learning models. These features can be turned on and turned off within the CMakeLists.txt file in the main MoDELib2 directory. Open the file and go to the lines

```
$ option(USE_PYBIND11 "enables_pybind11" ON)
$ option(BUILD_TOOLS "builds_tools" ON)
```

If *USE_PYBIND11* is turned off pybind11 will not be used to embed python into the MoDELib2 executables, and the standard tools microstructureGenerator, DDomp and DDqt are built. If both *USE_PYBIND11* and *BUILD_TOOLS* are turned off then only the python library of MoDELib2 will be built.

3.2.1 SDK version for Mac

Fing Xcode SDK version

```
$ xcrun --sdk macosx --show-sdk-path
```

Before cmake .. command run the following in the terminal with the correct SDK version

```
$ export LDFLAGS=
$ -L/Library/Developer/CommandLineTools/SDKs/MacOSX26.5.sdk/usr/lib; LDFLAGS="$LDFLAGS
$ -L/Library/Developer/CommandLineTools/SDKs/MacOSX26.5.sdk/usr/lib/system"
```

3.2.2 Python and pybind11 mismatch

If having issues with the correct python and pybind11 versions matching then you can run something similar to the following when running the cmake command to point to the correct python and pybind11 directories

```
$ cmake ..
$ -DPython3_EXECUTABLE=/opt/local/bin/python3.11
$ -DPython3_EXECUTABLE=/opt/local/bin/python3.11
$ -Dpybind11_DIR=/opt/local/bin/python3.11 -m pybind11 --cmakedir)
```

Chapter 4

Running MoDELib2

One chapter with pybind11, one section without pybind11, same tutorials but run with each version

Once compiled, go to the location of the folder MoDELib2 which you wish to run simulations and take note of the path to the folder of the desired tutorial. An example of this is by performing the following terminal commands.

```
$ cd MoDELib2/tutorials/$FolderName$/  
$ pwd
```

After taking note of the path, we can start to run our first MoDELib2 tutorial in the desired directory.

4.1 Input Files

Within the simulation folder you have opened you will see a python script called **generateInputFiles.py**. This script will create all the necessary input files needed to run a simulation in MoDELib2. The script will make local copies of the files included in the Library directory (*/MoDELib2/Library/*) and copy them into the simulation folder (*/MoDELib2/tutorials/\$FolderName\$/*). This script can be run with by invoking python with the following command while inside a tutorial folder (invoking python may change depending upon python set up):

```
# python3 generateInputFiles.py
```

This will create an “inputFiles” folder as well as an “evl” and “F” folder. The inputFiles folder will contain all inputs to the simulation, such as material, initial microstructure, and simulation parameters meanwhile the evl and F folders will contain the output data. For example, the F folder will contain data related to plastic strain, strain rates, glissile dislocation density, etc, for all time steps. The evl folders will contain specific information from each time step with nodal information such as node ID, velocities, positions and loop information such as burgers vector, area, etc. Within the “inputFiles” folder now, “Polycrystal.txt” was created is to select things such as domain size, crystal rotation, and mesh size. “DD.txt” is where one can control simulation parameters such as time step integration method, desired physics to include, as well as what data gets output to evl and F files. “ElasticDeformation.txt” is where the user can specify external displacement or loads to the simulation domain. The material files are where irradiation defect properties are stored as well as energetic information for dislocation motion. Once the simulation has been properly set up to the desired microstructure, dynamics parameters, and material parameters, simulations can now be run by creating the microstructure, running the dynamics, and finally visualizing the results with **MicrostructureGenerator**, **DDomp**, and **DDqt**, respectively. We direct you to choose one of the tutorials included in Chapter 5 to run a simulation. A more general use of the tools is outlined in the remainder of the section. If it is your first time running the newest version of MoDELib2 it is recommended to follow a tutorial. A general use of the tools is outlined below.

4.2 MicrostructureGenerator

The steps to modify a crystal structure and run MicrostructureGenerator are provided in the following list:

Step 1: Specify the dislocation configuration you wish to inject into the crystalline microstructure. All available dislocation configurations posted in Microstructure directory in the Library. Any modifications to the

dislocation configurations should be conducted to the generateInputFiles.py script or within the InputFiles folder once they have been created, and not the Library.

Step 2: Once the input files have been created, copy the path of the desired tutorial or folder and travel to the build directory of **MicrostructureGenerator**. For example, the **MicrostructureGenerator** can be run using the following syntax:

```
# cd Modelib2/build/tools/MicrostructureGenerator/  
# ./microstructureGenerator MoDELib2/tutorials/$FolderName$
```

Step 3: Under the evl directory in the tutorial folder, the .txt file evl_0 will be written using the configuration specified in the input files. This concludes the function of **MicrostructureGenerator**

4.3 DDomp

The steps to calculate the dislocation trajectory throughout the simulation are provided in this list. To run dynamics, use executable **DDomp**.

Step 1: Specify the desired loading condition within ElasticDeformation.txt. Here, you can specify whether a stress σ , stress rate $\dot{\sigma}$, strain ε , or strain rate $\dot{\varepsilon}$. The MachineStiffnessRatio at the bottom determines whether or not your applied loading is controlled primarily by applied stress or strain. Specify dislocation quadrature point density, time-step information, etc within the DD.txt input file.

Step 2: Once the input files have been created, copy the path of the desired tutorial or folder and travel to the build directory of **DDomp**. For example, **DDomp** can be run using the following syntax:

```
# cd Modelib2/build/tools/DDomp/  
# ./DDomp MoDELib2/tutorials/$FolderName$
```

DDomp will begin computing the dislocation trajectory within the microstructure. MoDELib2 performance scales with the number of sample points as well as the box size and number of periodic images in each axial direction. To speed up simulations, investigate the use of subcycling bins for approximating results. Within the evl directory sequential files (i.e. evl_1.txt) will automatically generate with details about the dislocation trajectory at discrete timesteps as specified in DD.txt. **DDomp** will populate local stress information, as well as kinetic and energetic information about the dislocation quadrature points.

4.4 DDqt

DDqt is the visualization tool used internally within MoDELib. MoDELib takes advantage of the open source VTK and QT libraries in order to provide seamless interpretation of simulation data. The process for utilizing **DDqt** is slightly different than the other executable functions, and is outlined below:

Step 1: Travel to the **DDqt** build directory. Call the executable with **./DDqt** as follows (The folder path is not needed here as was needed for MicrostructureGenerator and DDomp):

```
# cd Modelib2/build/tools/DDqt/  
# ./DDqt
```

Step 2: A new window will appear. In the new window, click on the file tab, and direct the filepath to the tutorial folder containing the data you wish to visualize. Note that the evl directory must contain evl_0.txt in order to see a dislocation configuration.

There are a number of tabs available for you to augment the visualization of the dislocation configuration. Current capabilities include visualization of Peach-Koehler forces, stacking fault forces, among other tools. To visualize sequential timesteps, utilize the arrows provided beside the step number over the tools seen on the left-hand side of the **DDqt** window. The number in the center indicates the timestep of the currently visualized file, while the number beside the double arrow indicates how many timesteps to jump forward/backward.

Chapter 5

Simulation Tutorials

5.1 Periodic Simulations

5.1.1 Dislocation Dipoles

A dislocation dipole consists of two dislocations with opposite Burgers vectors, typically separated by a small distance in a glide plane. These dipoles form naturally in materials due to thermal fluctuations, applied stress, or interactions with obstacles. Dipole simulations provide controlled test cases for validating DDD models against theoretical predictions and experimental observations. As a first tutorial with MoDELib2, we will perform a simulation of dislocation dipoles in Tungsten. We will first look at the `generateInputFiles.py` script to understand the simulation inputs. The first section of the python script will copy the **DD.txt** from `/MoDELib2/Library/` into the `inputFiles` folder. The first few lines of the python script contained in the Tungsten Dipoles tutorial folder look as such:

```
# setInputVariable('inputFiles/'+DDfile,'useFEM','0')
# setInputVariable('inputFiles/'+DDfile,'useDislocations','1')
# setInputVariable('inputFiles/'+DDfile,'useInclusions','0')
# setInputVariable('inputFiles/'+DDfile,'useElasticDeformation','1')
# setInputVariable('inputFiles/'+DDfile,'useClusterDynamics','0')
```

Where we are instructing MoDELib2 to use dislocations as well as elastic deformation without the FEM or cluster dynamics modules. After specifying the type of simulations desired the next lines of the python script look as such:

```
# setInputVariable('inputFiles/'+DDfile,'Nsteps','1000') # number of simulation steps
# setInputVector('inputFiles/'+DDfile,'periodicImageSize',np.array([8,8,8]),'')
# setInputVariable('inputFiles/'+DDfile,'EwaldLengthFactor','4')
# setInputVariable('inputFiles/'+DDfile,'timeSteppingMethod','fixed') # adaptive or fixed
# setInputVariable('inputFiles/'+DDfile,'dtMax','2')
# setInputVariable('inputFiles/'+DDfile,'dxMax','1') # max nodal displacement
# setInputVariable('inputFiles/'+DDfile,'use_velocityFilter','0') # don't filter velocity
# setInputVariable('inputFiles/'+DDfile,'useSubCycling','0')
```

Where we first can control the number of simulation steps with `Nsteps`, currently set to 1000. For periodic simulations we can input the number of images in each direction, as well as the Ewald length factor currently set to 8 images in each direction, with an ewald length factor of 4. The Ewald length factor controls the effective range of interactions by determining how quickly the real-space stress field decays. If you have 8 periodic images, and you set an Ewald length factor of 4, then the stress field will effectively converge at a cutoff distance of 4 times the characteristic system size. This means that only dislocations within that range (i.e., a fraction of the total periodic images) will significantly contribute to the stress field. We next have options to control the time stepping in MoDELib2, with an option of “fixed” or “adaptive”. The fixed time step is controlled by the input “`dtMax`” which is in code units. To find the time step in seconds please multiple the input number by b/cs , where b is the burgers vector and cs is the shear wave speed of the material. For example for tungsten $b/cs = 9.412187058138576e-14$, and with a `dtMax` of 2, the time step of each simulation step will be ≈ 0.19 ps. The adaptive time step is controlled by the input “`dxMax`”, which signifies the maximum nodal displacement in burgers vectors, therefore for a `dxMax` of 1 the maximum nodal displacement is 1 burgers vector, which is 0.2722 nanometers for Tungsten. It should be noted that MoDELib2 will always take the smallest time step between

the two, so if an adaptive time step is desired, it is recommended to place a large `dtMax`, to ensure the adaptive timestep is used. Next there are two other control parameters “`use_velocityFilter`” and “`useSubCycling`”. The velocity filter will ensure all dislocation node velocities remain within 75% of the shear wave speed of the material. SubCycling can be used to bin velocities and reduce computational costs, recommended for larger scale simulations. The final section of input parameters for `DD.txt` look as such:

```
# setInputVariable('inputFiles/'+DDfile,'glideSolverType','Galerkin') # type of glide solver
# setInputVariable('inputFiles/'+DDfile,'climbSolverType','none') # type of clim solver
# setInputVariable('inputFiles/'+DDfile,'quadPerLength','0.1')
# setInputVariable('inputFiles/'+DDfile,'alphaLineTension','1') # dimensionless scale factor
# setInputVariable('inputFiles/'+DDfile,'remeshFrequency','0') # DD interactions
# setInputVariable('inputFiles/'+DDfile,'Lmin','10') # min segment length (Burgers vectors)
# setInputVariable('inputFiles/'+DDfile,'Lmax','15') # max segment length (Burgers vectors)
# setInputVariable('inputFiles/'+DDfile,'outputFrequency','10')
# setInputVariable('inputFiles/'+DDfile,'outputQuadraturePoints','1') # output quadrature data
```

Where the `glideSolverType` and `climbSolverType` define the numerical methods used to solve for dislocation glide and climb motion. `glideSolverType`= “Galerkin” specifies that the Galerkin method will be used to solve for the glide motion of dislocations. `climbSolverType`=“none” indicates that no explicit climb mechanism is considered in this simulation. The climb solvers can be included if diffusion-driven dislocation motion is of interest, as outlined in the next tutorial. The “`quadPerLength`” parameter controls the quadrature resolution for segment interactions. A value of 0.1 means that quadrature points are spaced at 0.1 times the characteristic segment length. The length is expressed in Burgers vectors (b), meaning a value of 0.1 implies one quadrature point every 10 Burgers vectors. This affects the accuracy of force calculations along curved dislocation segments. The “`alphaLineTension`” is a dimensionless scale factor for line tension effects. A value of 1 means that default material-dependent line tension values are used, which influences the force balance and stability of dislocation segments. “`remeshFrequency`” controls how often the dislocation network is remeshed (i.e., how frequently segment sizes are adjusted for numerical stability and accuracy). A value of 0 means no re-meshing, so segment lengths remain fixed. If re-meshing is enabled, “`Lmin`” and “`Lmax`” define the minimum and maximum segment lengths (in units of Burgers vectors) for discretizing the dislocation network. `Lmin` = 10 means that dislocation segments will not be refined below 10 Burgers vectors in length. `Lmax` = 15 ensures that segments do not exceed 15 Burgers vectors, preventing overly coarse discretization. The “`outputFrequency`” defines how often the simulation outputs data. A value of 10 means that simulation outputs are written every 10 time steps in the `evl` folders. This balances storage efficiency and post-processing resolution. Finally “`outputQuadraturePoints`” ensures that quadrature data is included in the output files, which can be useful for post-processing stress fields, forces, and other detailed calculations. This information is stored in the “`ddAux`” files contained within the `evl` folder.

The material properties required for the simulation are stored in a file named `W.txt`. This file is initially copied from the `Library/Materials` directory into the simulation’s `inputFiles` folder. If necessary, modifications can be made to this local copy. The python script contains the following lines:

```
# materialFile = 'W.txt';
# setInputVariable('inputFiles/' + materialFile, 'dislocationMobilityType', 'default')
# setInputVariable('inputFiles/' + materialFile, 'enabledSlipSystems', 'full<111>{110}')
```

In this section of the script the **material file** is copied from the library and stored in the `inputFiles` directory. The **dislocation mobility type** is set to `default`, which determines how dislocations move in response to stress. The **enabled slip systems** are set to `full<111>{110}`, meaning that all {110} planes are active with <111> Burgers vectors, which are typical for BCC materials like Tungsten.

Next the elastic deformation settings are stored in the file `ElasticDeformation.txt`. Similar to the material file, this file is copied from the `Library/ElasticDeformation` directory into the `inputFiles` folder. The python script contains the following lines:

```
# elasticDeformationFile = 'ElasticDeformation.txt';
# setInputVector('inputFiles/' + elasticDeformationFile, 'ExternalStress0', np.array([0.0,
    0.0, 0.0, 0.0, 0.01, 0.0]), 'applied_stress')
```

The `ElasticDeformation.txt` file allows for the specification of external stress and strain conditions. The available options include:

- **External Stress:** Specifies the applied stress tensor in ****Voigt notation (11, 22, 33, 12, 23, 13)****.

```
# ExternalStress0 = 0.0 0.0 0.0 0.0 0.01 0.0; # Initial applied stress
# ExternalStressRate = 0.0 0.0 0.0 0.0 0.0 0.0; # Rate of change of stress
```

- **External Strain:** Specifies strain control conditions.

```
# ExternalStrain0 = 0.0 0.0 0.0 0.0 0.0 0.0; # Initial strain state
# ExternalStrainRate = 0.0 0.0 0.0 0.0 0.0 0.0; # Rate of change of strain
```

- **Stiffness Ratio:** Controls the boundary conditions between pure stress and pure strain control.

```
# stiffnessRatio = 0.0 0.0 0.0 0.0 0.0 0.0;
```

- `stiffnessRatio = 0` enforces ****pure stress control****. - `stiffnessRatio = 1e20` (or a very large value) enforces ****pure strain control****.

Chapter 6

Additional Features

6.1 Mesh Generation

Generating meshes for use with MoDELib2 can be done with `neper` (<https://www.neper.info/>) and `gsmh` (<https://www.neper.info/>). Once `neper` and `gsmh` are installed a tessellation file can be created by running the following commands:

```
# export OMP_NUM_THREADS=6
# neper -T -n 50 -domain "cube(1,1,1)" -morpho graingrowth -morphoptistop "itermax=3000" -o
  Polycrystal.tess
```

which will create a finite unit polycrystal with 50 grains. Creating tessellations allows for parallelization. For periodic simulations one can add the command “-periodicity _”

```
# neper -T -n 50 -domain "cube(1,1,1)" -morpho graingrowth -morphoptistop "itermax=3000"
  -periodicity x,y -o Polycrystal.tess
```

Next to mesh the tessellation, there are a number of options. For example to directly create the mesh file, one can invoke `gsmh`, to create a `.msh` file:

```
# export OMP_NUM_THREADS=1
# neper -M -gsmh /Applications/Gmsh.app/Contents/MacOS/gmsh Polycrystal.tess
```

which will create the `.msh` file directly compatible with MoDELib2 simulations. Meshing must be on a single thread. An example of a non-periodic, and periodic meshes visualized with DDqt are shown in Fig. 6.1. Currently MoDELib2 is being developed for use with fully periodic/mixed periodic meshes

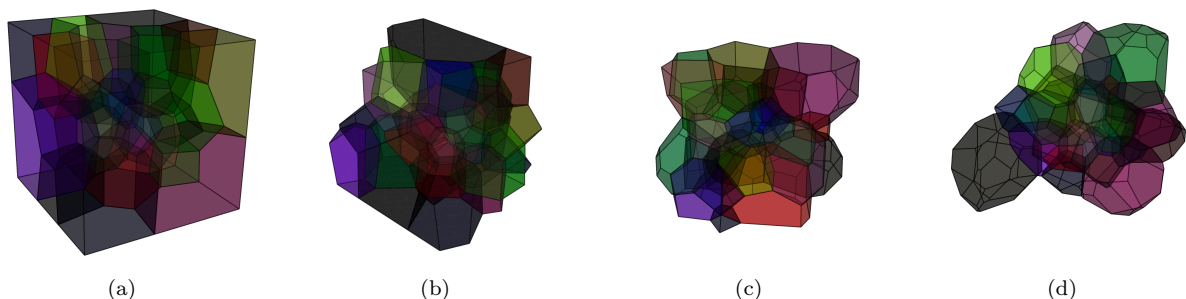


Figure 6.1: (a) Non-periodic mesh. (b) Periodic in one direction. (c) Periodic in two directions. (d) Periodic in three directions.

Chapter 7

Understanding the Code

SRC Dircetory

The src directory [`~/MoDELib/MoDELib/src/`] contains all of the .cpp C++ files which drive the simulation's functionality. As MoDELib is open source, you can modify certain elements of the code (carefully) and recompile the affected executable to tailor aspects of your simulation. MoDELib makes use of C++ smart pointers to reference data, and there are a number of defined templates in the model namespace. Getting acclimated to the format of each script can take time, but the naming convention of each file in each subdirectory ought to provide some indication of the purpose of each script's functionality.

INCLUDE Directory

The include directory contains all of the .h files which define all the functions utilized in MoDELib. The file tree is identical to what can be found in the src directory. If modifying any of the .cpp files utilized in the functionality of MoDELib, be sure to appropriately modify its respective header file.

Updating your MoDELib2 Repository

As MoDELib2 is a git repository, one has access to all typical git commands used to update local versions of the software from the main (protected) repository. Before updating, it is recommended that you save a version of your local MoDELib2 separately for reference.

Step 1: (Optional) Copy MoDELib2 directory to separate location for reference

Step 2: Run command "**git stash**" to maintain any local changes to the directory (files modified in newer release versions will still be overwritten)

Step 3: Run command "**git pull**" to update the repository with newer file versions

Step 4: Recompile all executable functions.

Appendix A

Text Editors

XCode and TextEdit

It is recommended to use XCode when editing or reading header files or source files from MoDELib2, but it can also be helpful to read files such as input files and material files with the built in *TextEdit* application on MacOS. To change the default application for opening specific files, such as ".txt" files, first `cntl+click` or left click a file, "test.txt" for example. Click *Get Info*, and go to the drop down menu *Open With*. In this menu the software for opening .txt files can be chosen as *TextEdit*; make sure to specify for all files of type ".txt". It is then recommended to follow the same procedure for ".h" and ".cpp" files, to be opened with Xcode. If you prefer to use terminal text editors, a brief tutorial on Vim is shown in the next section.

Vim

In this quickstart guide, we utilize a terminal text editor called 'Vim.' It is a highly extensible and programmable text editor used by many people. Vim can be installed with MacPorts using the following line in the terminal

```
$ sudo port install vim
```

We highly recommend that you learn the basic commands of using Vim before proceeding to the next section of this guide. To use Vim first open the Vim text editor,

```
$ vim
```

Then to open a file of the type "test.txt" in the current directory with Vim,

```
$ vim test.txt
```

Inside of Vim, press the following key to enter "edit" mode,

```
$ i
```

and the file test.txt can now be edited. Inside of Vim, press the following key to go back to "normal" mode from "edit" mode after changes to the file have been made,

```
$ ESC
```

Inside of Vim, press the following key to activate command prompt (must be in normal mode),

```
$ :
```

Type in the following command to exit Vim and save the changes made to the document,

```
$ :wq
```

Appendix B

Web Addresses for Software

1. MacPorts:
<https://www.macports.org/install.php>
2. Boost:
<https://github.com/boostorg/cmake?tab=readme-ov-file>
3. CMake:
<https://cmake.org/download/>
4. VTK:
<https://vtk.org/download/>
5. Qt:
<https://www.qt.io/download/>
6. MoDELib:
<https://github.com/giacomopo/MoDELib2/>